

Knowledge Neurons in Pretrained Transformers

Damai Dai^{†‡*}, Li Dong[‡], Yaru Hao[‡], Zhifang Sui[‡], Baobao Chang[‡], Furu Wei[‡]

[†]MOE Key Lab of Computational Linguistics, Peking University

[‡]Microsoft Research

{daidamai, szf, chbb}@pku.edu.cn

{lidong1, yaruhao, fuwei}@microsoft.com

Abstract

Large-scale pretrained language models are surprisingly good at recalling factual knowledge presented in the training corpus (Petroni et al., 2019; Jiang et al., 2020b). In this paper, we present preliminary studies on how factual knowledge is stored in pretrained Transformers by introducing the concept of *knowledge neurons*. Specifically, we examine the fill-in-the-blank cloze task for BERT. Given a relational fact, we propose a knowledge attribution method to identify the neurons that express the fact. We find that the activation of such knowledge neurons is positively correlated to the expression of their corresponding facts. In our case studies, we attempt to leverage knowledge neurons to edit (such as update, and erase) specific factual knowledge without fine-tuning. Our results shed light on understanding the storage of knowledge within pretrained Transformers. The code is available at <https://github.com/Hunter-DDM/knowledge-neurons>.

1 Introduction

Large-scale pretrained Transformers (Devlin et al., 2019; Liu et al., 2019; Dong et al., 2019; Clark et al., 2020; Bao et al., 2020) are usually learned with a language modeling objective on large-scale corpora, such as Wikipedia, where exists oceans of factual knowledge. Pretrained language models naturally play as a free-text knowledge base by predicting texts (Bosch et al., 2019). Petroni et al. (2019) and Jiang et al. (2020b) probe factual knowledge stored in pretrained language models by fill-in-the-blank cloze queries. The evaluation shows that pretrained Transformers have a strong ability to recall factual knowledge without any fine-tuning. Roberts et al. (2020) use closed-book question answering to show that the larger a model is, the more knowledge it can store. However, most previous work focuses on evaluating the overall accuracy of

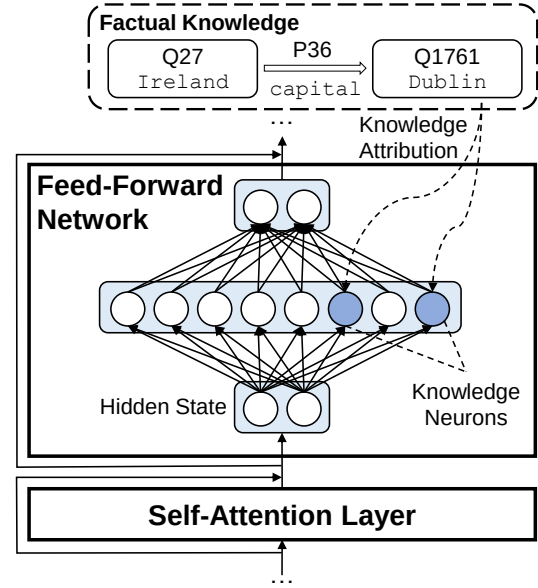


Figure 1: Through knowledge attribution, we identify knowledge neurons that express a relational fact.

text-form knowledge prediction. In this paper, we attempt to look deeper into pretrained Transformers and investigate how factual knowledge is stored.

As shown in Figure 1, we propose a knowledge attribution method to identify the neurons that express a relational fact, where such neurons are named *knowledge neurons*. Specifically, we view feed-forward network (i.e., two-layer perceptron) modules in Transformer as key-value memories (Geva et al., 2020). For the example in Figure 1, the hidden state is fed into the first linear layer and activates knowledge neurons; then, the second linear layer integrates the corresponding memory vectors. The key-value-memory nature (Geva et al., 2020) inspires us to propose the knowledge attribution method, which identifies knowledge neurons in feed-forward networks by computing the contribution of each neuron to the knowledge prediction.

Extensive analysis shows that the activation of the identified knowledge neurons is positively correlated to the knowledge expression, which shows

*Contribution during internship at Microsoft Research.

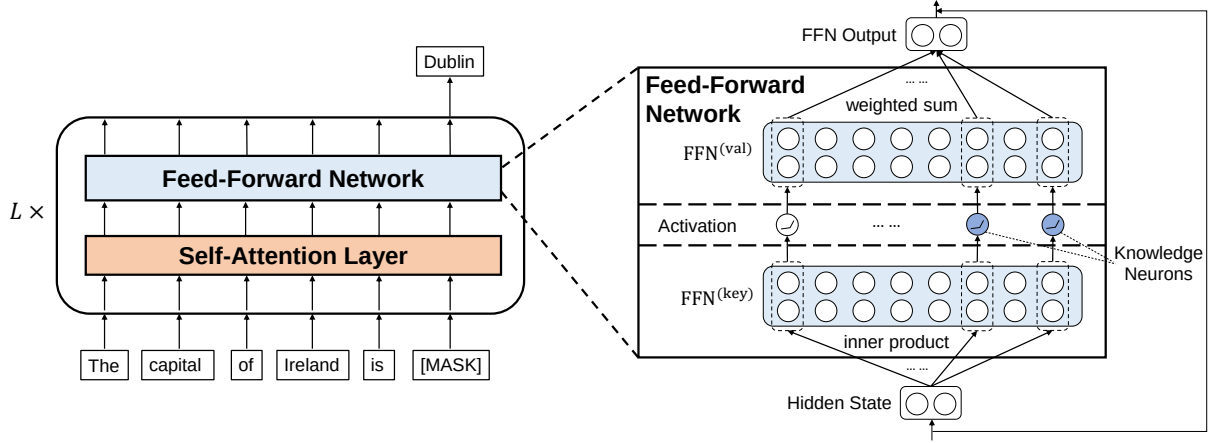


Figure 2: Illustration of how an FFN module in a Transformer block works as a key-value memory. The first linear layer $\text{FFN}^{(\text{key})}$ computes intermediate neurons through inner product. Taking the activation of these neurons as weights, the second linear layer $\text{FFN}^{(\text{val})}$ integrates value vectors through weighted sum. We hypothesize that knowledge neurons in the FFN module are responsible for expressing factual knowledge.

the effectiveness of the proposed knowledge attribution method. First, suppressing and amplifying knowledge neurons notably affects the expression of the corresponding knowledge. Second, we find that knowledge neurons of a fact tend to be activated more by corresponding knowledge-expressing prompts. Third, given the knowledge neurons of a fact, the top activating prompts retrieved from open-domain texts usually express the corresponding fact, while the bottom activating prompts do not express the correct relation.

In our case studies, we try to leverage knowledge neurons to explicitly edit factual knowledge in pretrained Transformers without any fine-tuning. We present two preliminary studies: updating facts, and erasing relations. After identifying the knowledge neurons, we perform a knowledge surgery for pretrained Transformers by directly modifying the corresponding parameters in feed-forward networks. Such surgery shows promising results, keeping a moderate influence on other knowledge.

Our contributions are summarized as follows:

- We introduce the concept of *knowledge neurons* and propose a knowledge attribution method to identify the knowledge neurons that express specific factual knowledge in the fill-in-the-blank cloze task.
- We conduct both qualitative and quantitative analysis to show that knowledge neurons are positively correlated to knowledge expression.
- We present preliminary studies of leveraging knowledge neurons to edit factual knowledge

in Transformers, even without any fine-tuning.

2 Background: Transformer

Transformer (Vaswani et al., 2017) is one of the most popular and effective NLP architectures. A Transformer encoder is stacked with L identical blocks. Each Transformer block mainly contains two modules: a self-attention module, and a feed-forward network (abbreviated as FFN) module. Let $X \in \mathbb{R}^{n \times d}$ denote the input matrix, two modules can be formulated as follows:

$$Q_h = XW_h^Q, K_h = XW_h^K, V_h = XW_h^V, \quad (1)$$

$$\text{Self-Att}_h(X) = \text{softmax}(Q_h K_h^T) V_h, \quad (2)$$

$$\text{FFN}(H) = \text{gelu}(HW_1) W_2, \quad (3)$$

where $W_h^Q, W_h^K, W_h^V, W_1, W_2$ are parameter matrices; $\text{Self-Att}_h(X)$ computes a single attention head; H , the hidden state, is given by projecting the concatenation of all heads; gelu denotes the GELU activation function (Hendrycks and Gimpel, 2016). For simplicity, we omit the scaling factor in self-attention and the bias terms.

Connections Between Self-Attention and FFN

Comparing Equation (2) and Equation (3), we notice that the formula of $\text{FFN}(\cdot)$ is quite similar to $\text{Self-Att}(\cdot)$, except the activation function gelu in FFN and softmax in self-attention. Thus, similar to the query-key-value mechanism in self-attention, it is reasonable to regard the input of the FFN as a query vector, and two linear layers of the FFN as keys and values, respectively. Similar observations are also described in (Geva et al., 2020).

3 Identifying Knowledge Neurons

Similar to (Geva et al., 2020), we view FFNs in Transformer as key-value memories as illustrated in Figure 2. We hypothesize that factual knowledge is stored in FFN memories and expressed by *knowledge neurons*. In this section, we propose a knowledge attribution method and a refining strategy to identify these knowledge neurons.

3.1 Knowledge Assessing Task

We employ the fill-in-the-blank cloze task to assess whether a pretrained model knows a fact. Following Petroni et al. (2019), each relational fact is in the form of a triplet $\langle h, r, t \rangle$, where h is the head entity, t is the tail entity, and r is the relation between them. Given a fact, pretrained models answer the cloze query x that expresses the fact but leaves the tail entity as a blank. For example, given the fact $\langle \text{Ireland}, \text{capital}, \text{Dublin} \rangle$, a possible query is “The capital of Ireland is ___”. We also call the query a *knowledge-expressing prompt*. Petroni et al. (2019) describe that a model knows a fact if it can predict the correct answer. In this paper, rather than just examining the model outputs, we identify the specific knowledge neurons that express factual knowledge.

3.2 Knowledge Attribution

Inspired by Hao et al. (2021), we propose a knowledge attribution method based on integrated gradients (Sundararajan et al., 2017). Our method can evaluate the contribution of each neuron to knowledge predictions. In this paper, we examine FFN intermediate neurons for the masked token, where the answer is predicted.

Given an input prompt x , we first define the model output $P_x(\hat{w}_i^{(l)})$ as the probability of the correct answer predicted by a pretrained model:

$$P_x(\hat{w}_i^{(l)}) = p(y^* | x, w_i^{(l)} = \hat{w}_i^{(l)}), \quad (4)$$

where y^* denotes the correct answer; $w_i^{(l)}$ denotes the i -th intermediate neuron in the l -th FFN; $\hat{w}_i^{(l)}$ is a given constant that $w_i^{(l)}$ is assigned to.

In order to calculate the attribution score of a neuron $\text{Attr}(w_i^{(l)})$, we gradually change $w_i^{(l)}$ from 0 to its original value $\bar{w}_i^{(l)}$ calculated by the pretrained model, and meanwhile integrate the gradients:

$$\text{Attr}(w_i^{(l)}) = \bar{w}_i^{(l)} \int_{\alpha=0}^1 \frac{\partial P_x(\alpha \bar{w}_i^{(l)})}{\partial w_i^{(l)}} d\alpha, \quad (5)$$

where $\frac{\partial P_x(\alpha \bar{w}_i^{(l)})}{\partial w_i^{(l)}}$ calculates the gradient of the model output with regard to $w_i^{(l)}$. Intuitively, as α changes from 0 to 1, by integrating the gradients, $\text{Attr}(w_i^{(l)})$ accumulates the output probability change caused by the change of $w_i^{(l)}$. If the neuron has a great influence on the expression of a fact, the gradient will be salient, which in turn has large integration values. Therefore, the attribution score can measure the contribution of the neuron $w_i^{(l)}$ to the factual expressions.

Directly calculating continuous integrals is intractable. We instead use Riemann approximation $\tilde{\text{Attr}}(w_i^{(l)}) = \frac{\bar{w}_i^{(l)}}{m} \sum_{k=1}^m \frac{\partial P_x(\frac{k}{m} \bar{w}_i^{(l)})}{\partial w_i^{(l)}}$, where $m = 20$ is the number of approximation steps. With the attribution algorithm, we can identify a coarse set of knowledge neurons whose attribution scores are greater than a threshold t .

3.3 Knowledge Neuron Refining

In order to identify knowledge neurons more accurately, we further propose a refining strategy. Besides “true-positive” knowledge neurons that express factual knowledge, the coarse set of knowledge neurons may contain “false-positive” knowledge neurons that express other information (e.g., syntactic or lexical information). The refining strategy aims to filter out these “false-positive” neurons.

For different prompts corresponding to the same fact, we hypothesize that they share the same set of “true-positive” knowledge neurons, since they express the same factual knowledge. Meanwhile, we hypothesize that they do not share the “false-positive” knowledge neurons as long as the prompts are diverse enough. Therefore, given multiple diverse prompts, we can refine the coarse set of knowledge neurons by retaining only neurons that are widely shared among these prompts.

Specifically, given a relational fact, the complete process to identify its knowledge neurons is described as follows: (1) produce n diverse prompts; (2) for each prompt, calculate the knowledge attribution scores of neurons; (3) for each prompt, retain the neurons with attribution scores greater than the attribution threshold t , obtaining the coarse set of knowledge neurons; (4) considering all the coarse sets together, retain the knowledge neurons shared by more than $p\%$ prompts.

Relations	Template #1	Template #2	Template #3
P176 (manufacturer)	[X] is produced by [Y]	[X] is a product of [Y]	[Y] and its product [X]
P463 (member_of)	[X] is a member of [Y]	[X] belongs to the organization of [Y]	[X] is affiliated with [Y]
P407 (language_of_work)	[X] was written in [Y]	The language of [X] is [Y]	[X] was a [Y]-language work

Table 1: Example prompt templates of three relations in PARAREL. [X] and [Y] are the placeholders for the head and tail entities, respectively. Owing to the page width, we show only three templates for each relation. Prompt templates in PARAREL produce 253,448 knowledge-expressing prompts in total for 27,738 relational facts.

4 Experiments

4.1 Experimental Settings

We conduct experiments for BERT-base-based (Devlin et al., 2019), one of the most widely-used pre-trained models. It contains 12 Transformer blocks, where the hidden size is 768 and the FFN inner hidden size is 3,072. Notice that our method is not limited to BERT and can be easily generalized to other pretrained models. For each prompt, we set the attribution threshold t to 0.2 times the maximum attribution score. For each relation, we initialize the refining threshold $p\%$ (Section 3.3) as 0.7. Then, we increase or decrease it by 0.05 at a time until the average number of knowledge neurons lies in $[2, 5]$. We run our experiments on NVIDIA Tesla V100 GPUs. On average, it costs 13.3 seconds to identify knowledge neurons for a relational fact with 9 prompts.

4.2 Dataset

We examine knowledge neurons through the fill-in-the-blank cloze task based on the PARAREL dataset (Elazar et al., 2021). PARAREL is curated by experts, containing various prompt templates for 38 relations from the T-REx dataset (ElSahar et al., 2018). We show some example templates in Table 1. For each relational fact, we fill in the head entity in prompt templates and leave the tail entity as a blank to predict. In order to guarantee the template diversity, we filter out relations with fewer than 4 prompt templates and finally keep 34 relations, where each relation has 8.63 different prompt templates on average. These prompt templates produce 253,448 knowledge-expressing prompts in total for 27,738 relational facts.

4.3 Attribution Baseline

Our baseline method takes the neuron activation value as the attribution score, i.e., $\text{Attr}_{\text{base}}(w_i^{(l)}) = \bar{w}_i^{(l)}$, which measures how sensitive a neuron is to the input. After computing attribution scores, we follow the same pipeline to obtain the refined

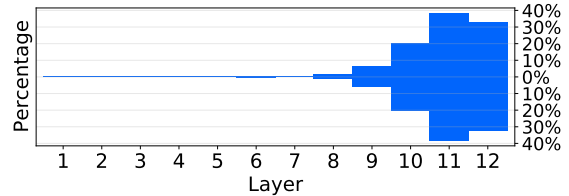


Figure 3: Percentage of knowledge neurons identified by our method in each Transformer layer.

Type of Neurons	Ours	Baseline
Knowledge neurons	4.13	3.96
$\cap$ of intra-rel. fact pairs	1.23	2.85
$\cap$ of inter-rel. fact pairs	0.09	1.92

Table 2: Statistics of knowledge neurons. $\cap$ denotes the intersection of knowledge neurons of fact pairs. “rel.” is the shorthand of relation. Our method identifies more exclusive knowledge neurons.

knowledge neurons. For a fair comparison, we employ the same method to choose the hyper-parameters t and $p\%$ for the baseline to ensure the average number of knowledge neurons for each relation lies in $[2, 5]$.

The method based on neuron activation is a reasonable baseline. It is motivated by FFNs’s analogy with the self-attention mechanism (as described in Section 2), because self-attention scores are usually used as a strong attribution baseline (Kovaleva et al., 2019; Voita et al., 2019; Hao et al., 2021).

4.4 Statistics of Knowledge Neurons

Figure 3 presents the layer distribution of knowledge neurons identified by our knowledge attribution method. We notice that most fact-related neurons are distributed in the topmost layers of pre-trained Transformers. The finding also agrees with Tenney et al. (2019) and Geva et al. (2020).

Table 2 shows statistics of knowledge neurons. On average, we identify 4.13 knowledge neurons for each relational fact using our knowledge attribution method, and 3.96 using the baseline method.